POORNIMA

Experiment -1

AIM :- understanding the history of software security based

The history of software security - Recognizing Web Application Security Threats. The history of software security, particularly in the context of recognizing web application security threats, is marked by the evolution of technology and the increasing sophistication of cyber threats. Below is a broad overview of key milestones and developments in the history of web application security :-

1. 1990's the emergence of the world wide web :- the world wide web became publicly accessible leading to the proliferation of websites and web applications - the focus during the period was on building the web with limited attention to security considerations.

2. late 1990s to early 2000s :- rise of common vulnerabilities as web application grew in complexity, security vulnerabilities become more apparent. common vulnerabilities such as buffer overflows, SQL injection and cross site scripting started to emerge the concept of input validation and sanitization was a crucial aspect of web application security

Page No.

POORNIMA

2002 :- the birth of owasp :- the open web application Security Project founded founded to provide resource and guidelines for improving web application security the owasp top ten a list of the most critical web application security risks. were introduced to provide common robust common vulnerabilities.

Mid 2000s :- proliferation of Web 2.0 and AJAX the advent of web 2.0 technologies and the use of Asynchronous JS and XML (AJAX) introduced new attack vectors Security researchers and collector began to exploit vulnerabilities related to the dynamic and interactive nature of these technologies.

late 2000 :- Increased focus on Secure Development Practices - organization started recognizing the importance of Integrating Security into the software development life cycle. Secure coding practices and tools became more powerful prevalent emphasizing the need to consider security from the design phase onward.

2010s :- evolution of threat landscape :- the threat landscape continued to evolve with the rise of mobile applications cloud computing and API's. Web Security Standards like HTTP Strict Transport Security (HSTS) and content Security Policy gained adoption to enhance protection against various.

POORNIMA

- Introduction of security standard bio
  - OWASP
  - CERT

- Common Web threats
  - SQL injection
  - cross side scripting
  - Cross Side Request Forgery (CSRF)
  - Broken Authentication
  - Security Misconfiguration

Tools :- Burp
- ~~OWASP~~ OWASP top 5 documentation

POORNIMA

Experiment-1(a)

Aim :- to explore the Analyze the difference btw Http & vs Https by using wireshark.

Installation of wireshark software

- open the web browser
- search wireshark download
- Select the windows installer according to your system configuration either 32-bit or 64-bit save the program and close the browser.
- Now open the software and follow the install instruction by accepting the license
- the wireshark is ready for use.

HTTP capture

1) Before start analysing any packet please turn off "Allow subdissectors to reassemble TCP streams" (Preference → Protocal → TCP) (The coil parsed TCP packets to split into multiple pdu unit

As you can see - I am HTTP so that the encryption will not be hidden behind TLS.

As you cannot see at the number 13 standard DNS resolution is happing

POORNIMA

in line numbers 7 you see the response we are getting back with full DNS resolution.

Now if you look at Packet number 8 i.e we get request of HTTP primarily used two command.

1. GET :- to retrive information
2. POST :- to send information (ex. e.g. when we submit some form we fill some data i.e. is POST)

Here i am trying to get classload.html via HTTP protocol 1.1 (the new version of protocol is now evelid. ie. 2.0)

Then at line numbers 5 we see the acknowledgment as well as line numbers 6 server was able to found that page and sed HTTP status code 200

if you want more info about HTTPS code
you will see some more info like for packet 6, like server type is Apache content type is HTML. how long the content length

Then you will see bunch of continuation that is due to TCP window where is as you dont get acknowledged Before and every packet.

and at that top some usual TCP handshake
capture HTTPS in Wireshark

Page No.

POORNIMA

1. launch wireshark and start an unfiltered capture session.
2. Minimize the wireshark window and open your browser.
3. Go to any secure website to get data.
4. return to wireshark and select any frame with encrypted data.
5. find 'Packet byte view' and look at 'Decrypted SSL' data. HTML should now visible.

Conclusion

HTTPS provide better security than HTTP due to following reasons.

- encryption : the use of SSL/TLS encryption in HTTPS ensure that the data being transmitted between the client and the server is secure and cannot be intercepted by an attacker.
- Authentication : HTTPS use digital certificates to authenticate the identity of the website, making it harder for attackers to impersonate a website and carry out a man-in-the-middle attack.
- integrity : HTTPS provide data integrity which ensure that the data being transmitted between the client and the server has not been modified or tampered with.
- SEO :- HTTPS is now a ranking signal for search engine and many can HTTPS site can improve their search engine ranking.

POORNIMA

Experiment 1(b)

Aim:- To Analyze the various security mechanism embedded with protocol by using wireshark.

Analyzing the security mechanism embedded in different protocols using wireshark involve examining network traffic to understand how various security features are implemented and whether they are effective Some common security mechanism embedded in different protocols that you can analyze using Wireshark.

1. Transport layer Security :- TLS / SSL is used to encrypt and secure data transmission over the network
-> in wireshark you can identify TLS / SSL traffic by looking at the protocol field, which should indicate TLS or SSL.
- you can analyze the TLS handshake process, certificate exchange and the encryption ciphers being used.

2. Secure Shell (SSH) :- SSH is a secure protocol for remote access and data exchange
- wireshark can capture SSH traffic and you can analyze the key exchange authentication method and encryption algorithms being used.

POORNIMA

IPSec (Internet Protocol Security):-
- IPSec is used to secure IP communication through encryption and authentication.
- Wireshark can capture and analyze IPSec packets to understand the negotiation of security association (SA's) encryption algorithms and authentication methods.

5. HTTP Secure
- HTTPS is a secure version of HTTP that uses TLS/SSL for encryption.
- Wireshark can capture HTTPS traffic and allow you to examine the encrypted content after decryption by providing the TLS keys.

6. Secure Socket Layer (SSL) VPN:-
- SSL VPN's create secure connections for remote access.
- Wireshark can capture SSL VPN traffic and allow you to analyze the SSL handshake, certificate exchange and data transmission.

7. Kerberos:-
- Kerberos is a network authentication protocol.
- You can capture and analyze Kerberos traffic in Wireshark to observe the ticket granting process authentication and encryption.

Page No.

POORNIMA

8. SNMPV3 (Simple Network Managed Protocol version 3):
- SNMP v3 include security feature such as authentication and encryption.
- wireshark can you help you understand how SNMPv3 is used to secure network managed.

9. DNSSEC (Domain Name System Security Extensions):
DNSSEC adds security to the DNS by signing DNS records. wireshark can be used to capture & DNSSEC - Signed DNS queries and response to validate the digital signatures.

10. OAuth and OAuth 2:
- OAuth is used for delegated authorization, often in web and API interactions
- you can capture OAuth flows and analyze the token exchange and authentication mechanism.

When analyzing security mechanism using wireshark, it's essential to focus on packet capture related to the specific protocol or technology you're interested in. look for indicators of encrypted authentication key exchange and other security factors to ensure the effectiveness and security of the implemented. Additionally pay attention to any unencrypted or potential vulnerable that may be present in the captured traffic.

Page No.....

POORNIMA

Experiment-2

Aim : Perform vulnerability testing on a web application.

1. installing ZAP

↳ you can download the latest version from the OWASP ZAP website for your operating system to install ZAP or refer to the ZAP docs for a more detailed installation guide

↳ once completed, follow the Prompts to install OWASP ZAP on your machine

2. Persisting a Session

Persisting a session in OWASP ZAP means that the session will be saved and can be used or reopened at a later time. This is useful if you to continue testing a website or application at a later time.

once you've OWASP ZAP, you will see a screen that looks like this. The prompt gives two options to persist the session. You can use the default to name the session based on the current timestamp or set your own and location.

Alternatively you can persist a session by going to file and choosing persist session give your session a

POORNIMA

a name and click on the Save button.

To run an automated scan, you can use the Quick Start Automated Scan option under the Quick Start tab. Enter the URL of the site you want to scan in the 'URL to attack' field and then click Attack.

4. Interpreting test results:
Interpreting test results in OWASP ZAP is vital to understand the scan findings and determine which issues require further investigation. Additionally, it can help to prioritize remediation efforts.

In OWASP ZAP, you can view alerts by clicking on the Alerts tab. This tab will show you a list of all the alerts that have been triggered during your testing. The alerts are sorted by risk level, with the highest risk alerts at the top of the list. OWASP ZAP will give details of the discovered vulnerabilities and suggestions on how you can fix them.

5. Viewing alerts and alert detail in OWASP ZAP is a way to see what potential security issues have been identified on a website. It can help security professionals understand what needs to be fixed to improve the app's security.

Page No.....

POORNIMA

If you cannot find your 'Alerts' tab you can access it via the view menu, along with other options available in OWASP ZAP once you have your Alerts tab you can navigate the various vulnerabilities discovered and explore the reports generated by OWASP ZAP.

5 Viewing alerts and alerts details in OWASP ZAP is a way to see what potential security issues have been identified on a website. It can help discover and understand vulnerabilities that need to be fixed to improve the overall security.

Page No.....

POORNIMA

Experiment-3

Aim:- To create simple REST API using Python for CURD operation

Procedure:- • install flask
• creating a simple REST API in Python for basic CURD (Create Update Read delete) operations typically involve using a web framework.

code:- 1. a install flask
(in terminal)
Pip install flask

2. Create a new file called appy
from flask import Flask, request, jsonify
app = flask (__name__)

data = [
{"id": 1", "name" : "item1", description": "this is the first item" },
{"id": 2, "name" "item 2", "description" :
"This is the second item"}
{"id" : 3, "name" : "item 3", description" : this the third item" }

POORNIMA

```
# Get (retrive all items)

@app.route("/items", methods=['GET'])
    def get_items():
        return jsonify(data)

# POST (Create a new item)
@app.route("/items", methods=['POST'])
  def create_item():
      item = request.get_json()
      data.append(item)
      return jsonify({"message": "Item created
                                  successfully"})

# Put (update an item)
  @app.route("/items/<int:item_id>", methods=['PUT'])
    def update_item(item_id):
        if 0 <= item_id < len(data):
            updated_item = request.get_json()
            data[item_id] = updated_item
            return jsonify({"message": "Item updated successfully"})
        else:
            return jsonify({"message": "Item not found"}), 404
```

POORNIMA

```
# delete
@app.route ('/items/<int:item_id>', methods = ['delete'])
def delete_item(item_id):
    if 0 <= item_id < len(data):
        del data[item_id]
        return jsonify ({"message": "item deleted
                                    successfully"})
    else:
        return jsonify ({"message": "item not found"}), 404

if __name__ == '__main__':
    app.run (debug = True)
```

POORNIMA

objective :- understand and implement each phase of the vulnerability Assessment lifecycle.

Theory :- the vulnerability Assessment lifecycle is a continuous process, not a one-time event. because new threats emerge daily, security professionals use this framework to ensure that an organization's defense remain strong it moves beyond just "finding" a bug to actively managing the risk it poses to the business

The Five core phases

1. Preparation (Scope & discovery) : defining what needs to be tested (e.g specific servers. IP ranges, or web pages) and identifying all active within that scope.

2. Vulnerability Scanning : using automated tools to probe the identified assets for known weakness misconfigurations or outdated software versions.

3. Analysis & Prioritization :- evaluating the scan results to remove false positive vulnerabilities are ranked using the common vulnerability Scoring (CVSS) to decide which one need fixing first.

4. Remediation : the process of patching the hole this involve updating software, changing configuration settings or

Page No.

POORNIMA

implementing fixes and it's

5. verificate :- Re-scanning the system to ensure the patches resolved and generating a final report for stakeholders.

Step-by-Step procedure

Step1 :- Scoping the target

- identify the target web application or server IP
- document the "Rule of engagement" (what time the test happens and what tools are allowed).

Step2 : Scanning for vulnerabilities

- launch an automated scanning like Nessus, open VAS or OWASP ZAP
- input the target IP/URL and start a full discovery" scan.
- Result the tool will generate a list of plugins or checks that failed.

Step 3 : Analyzing the Risk

- Review the generated report
- filter the findings by severity : critical, high, medium, low.
- verify a finding manualy (e.g. if the scanning says version 2.0 is vulnerable, check the server to see if it's actually running version 2.0).

Step 4 :- Creating a Remediation Plan
- For a critical vulnerability (like an unpatched SQL injection) document the specific path or code change required.
- Assign a deadline for fix.

Step 5 : final validation
- After the "developer" applies the fix, rerun the scan against
- Result :- the vulnerability should no longer appear in the "Active list of the report"

Result :- the final output of this experiment is a vulnerability Assessment Report. the document includes :-
- Executive Summary :- a high level view of the security posture
- Technical findings :- a table, showing the cvt ID, Severity Score and Affected Asset
- proof of concept :- Screenshots showing how a vulnerability could be identified.
- Remediation Status :- Confirmation that critical bugs have been closed.

POORNIMA

experiment -5

Objective :- Perform a security assessment of an Android App.

Basic theory :- Android security assessment differs from web security because the application code (the APK file) resides on the user's device this makes it susceptible to reverse engineering. Attackers look for hardcoded sensitive data insecure local storage and improper communication with backend services.

Key concepts :-
- Static Analysis :- examining the application source code, manifest files and resources without executing the app
- Dynamic Analysis :- testing the application while it is running to monitor network traffic and memory usage.
- Apk Structure :- An Android package contains the classes.dex AndroidManifest.xml (permissions and components) and resources.

Required tools :
JADX-GUI for decompiling APK files into readable java code
- MobSF (Mobile Security Framework) An Automated, all-in-one mobile application pentesting framework
- Burp Suite :- to intercept and analyze traffic between the app and the server
- ADB (Android Debug Bridge) :- to communicate with the device or emulator.

POORNIMA

Step-by-Step Procedure

Step1:- Preparation & Decompilation.
- obtain the target .apk file
- open the file in JADX-GUI to convert the Dalvik bytecode back into java source code.
- Result:- you can now browse the internal package structure and logic of the app.

Step2:- Manifest File Analysis
check AndroidManifest.xml for security misconfigurations:
- android:debuggable="true" : Allows attackers to attach a debugger to the process.
- android:allowBackup="true" : Allows application data to be backed up via ADB, potentially exposing sensitive files.
- permissions:- ensure the app doesn't request unnecessary permissions.

Step3:- Static Code Analysis
- Search the codebase for sensitive keywords: API_KEY, Secret, Password, SSL or http://
- check for insecure data storage

Step4:- Dynamic Analysis
- configure the Android device to use Burp Suite as a proxy
- install the Burp CA certificate on the device to intercept HTTPS traffic
- interact with the app (login, submit forms) and observe the

report to BAP.
result :- identify if the app sends sensitive data over unencrypted channel or if it lacks SSL pinning.

Steps :- logging Analysis
- use adb logcat to monitor the system logs while the app is running.
- check if the app is leaking sensitive information.

expected Result / Output :
1. a detailed assessment report that includes :
- decompiled source code snippets :- highlighting hardcoded credentials
- Vulnerability Table :- listing found issues such as "insecure data storage" or "weak SSL implementation".
- Remediation :- Recommendations for the developer such as using the Android Keystore System to encrypt data disabling debug code.

POORNIMA

experiment :- 6

Objective :- understand the impact and mechanics of social engineering attacks.

Basic-theory :- Social engineering is the art of manipulating individuals into divulging confidential information or performing actions that compromise security unlike technical attacks that target software vulnerabilities, social engineering targets the human element. which is considered the weakest link in the security chain.

Common techniques :-
- phishing :- sending fraudulent emails that appear to come from a reputable source to steal data like login credentials
- Smishing / vishing :- Phishing via SMS or voice calls
- pretexting :- creating a fabricated scenario (the pretext) to steal a victim's personal information
- Baiting :- Leaving malware-infected physical media (e.g. USB drives) for a victim to find and use

Page No.....

Required tools :-

1. Social-engineer toolkit (SET) : An open-source python based tool designed for penetration testing that features several powerful attack vectors.
- kali linux :- the operating system where set is pre-installed.
- target browser : a machine used to simulate the victim's interaction.

Step-by-Step procedure :

Step 1 :- launch the Social-engineer toolkit
- open the terminal in kali linux and type sudo setoolkit
- select 1) Social-engineering Attacks from the main menu.

Step 2 :- Select the Attack Vector
- Select 2) website attack vectors. this is commonly used to harvest credentials by cloning a legitimate website.
- Select 3) credential harvester Attack Method.

Step 3 : configure the site cloner
- Select 2) site cloner
- IP Address :- enter the IP address of your kali linux machine where the stolen data will be sent
- URL to clone :- enter a URL for a simple login page.

Step 4 : execute and capture
- once the site is cloned the toolkit will start a web server on your machine.
- from a victim machine, navigate to the IP address of the attacker machine in a web browser.
- enter a dummy username and password in the cloned login page.

Step 5 :- Analyze of Results
- Return to the attacker terminal. The SET toolkit will display the captured username and password in clear text as they are submitted.

Result :- 1. terminal output :- the terminal should show a HIT once the victim interacts with the page. followed by the specific form fields (username and password) captured from the fake site.

2. inference :- the experiment demonstrate that even if a system is technically secure a user can be tricked handing over their credentials through visual deception.

POORNIMA

• experiment - 7

Objective: detect to SQL injection and how to prevent it?

Basic theory :- SQL injection is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. it occurs when untrusted user is concatenated directly into a SQL query string rather than being handled as data. this allows an attacker to inject their own SQL commands potentially ~~pa~~ query access to sensitive data modify database records or administrative operations

Type of SQLi :-

• in-band (classic) :- the attacker uses the same communication channel to launch the attack and gather results

• inferential (Blind) :- the attacker observes the server's response (Boolean - based or Time-based) to infer data when the results aren't directly displayed.

• out-of-band : the attacker triggers the database to make an external network request

Required Tools

- Vulnerable web App : Setup of a custom PHP / MYSQL login page.
- Interception proxy : Burp Suite (to modify requests).
- Browser to interact with the application

Step-by-Step Procedure :

Part A :- the Attack (demonstration)

1. Identify an input field :- Navigate to login or a search bar that interacts with a database.

2. Test for vulnerability :- enter a single quote (') in the input field. If the application returns a database error

3. Bypass Authentication :- in a login username field, enter the following payload :
   - admin' or '1' = '1' --

4. Analyze the logic :- the server-side query becomes :
   Select * from users where username = 'admin' AND password = '...'
   '1' = '1' -- AND password = '...'
   - The '1' = '1' is always true and the -- comment out the rest the original query, allowing you to log in without a password.

POORNIMA

5. exfiltrate data (Union-based) : use UNION SELECT statement to find the number of column and extract table names or user credentials

Part B :- The Prevention (remediation)

The core of prevention is ensuring that user input is never executed as code.

1. Primary defense :- Prepared statements (with Parameterized Queries) instead of building a string, you define the SQL code first and then the user input as a parameter.
   - Vulnerable code : query = "SELECT * FROM users WHERE id=" + inputID
   - Secure code query - "Select * FROM users where id = ?";
     stmt.setInt(1, inputID);

2. input validation : use a 'whitelist' to only allow expected input

3. principle of least privilege :- ensure the database user account used by the web app has only the minimum permissions necessary

expected Result / Output

- the attacker successfully log in as an administrator user retrieve a list of database users without knowing a valid password.

- The prevention :- After implementing proper statement the same payload ('OR '1'='1') is treated as a literal string. the database for a user whose name is exactly that string fails to find it. and does not execute

POORNIMA

objective identify flaws in session and authentication handling

Basic theory :- Authentication is the process of verifying a user's identity while session management maintains that identity across multiple requests. Flaws in these areas are exploited because they can allow an attacker to hijack a user's session or impersonate them without needing a password.

Common flaws :-

- Predictable session ids :- when session tokens are generated using weak algorithms allowing an attacker to guess a valid ID
- Session fixation :- An attacker "fixes" a user's session ID to a known value before the user logs in
- insecure token storage :- storing session ids in URL strings or unencrypted cookies
- lack of proper invalidation :- session that remain active indefinitely even after the user closes the browser or logs out.

Required tools

- Burp Suite :- essential for inspecting cookies and token generation patterns
- Browser developer tools (F12) :- to view and modify cookies manually.
- OWASP ZAP :- for automated testing of session strength

Page No.

Step-by-Step procedure

Step1:- inspecting cookie Attributes

1. log in to a target application
2. open Burp Suite or Browser DevTools and navigate to the cookie tab.
3. check for the presence of security flags
   - HttpOnly prevents javascript from accessing the cookie
   - Secure :- ensure the cookie is only sent over HTTPS
   - SameSite :- reduces whether cookie are sent cross-site request
4. Result :- if these flags are missing the session is vulnerable to theft.

Step 2 :- testing for session fixation

1. Navigate to the login page and note the current session ID
2. log in to the application with valid credentials.
3. check the session ID again
4. Result :- if the session id remains the same before and after login, the application is vulnerable to session fixation.

Step 3 : Session token Analysis

1. log in and note your session token
2. perform a logout using the application "logout" button.

3. try to navigate back to protected page or manually resend a request with the old session token in burp Repeater
4. Result :- if the server still accepts the old token, the session was not properly destroyed on the server-side

Step 4 : Predictability Analysis (Using Burp Sequencer)

1. capture a series of session tokens
2. send the token to burp sequencer
3. run the analysis to test the randomness and entropy
4. result :- if the entropy is low, the session ID is predictable and can be brute-forced.

Prevention (Remediation)

- Regenerate Session ID's : Always issue a new session token immediately after a successful login
- Set Secure flags : ensure cookies session cookies are marked as HttpOnly and Secure
- Server-Side invalidation : ensure the server explicitly destroys the session record when the user log-out.
- implement timeouts : enforce both idle timeouts.

expected Result / output

1. captured evidence : Screenshot showing cookie missing secured flags
2. vulnerability confirmation : A demonstrable color can old cookie token is used unsuccessful to access a private page after a supposed logout.
Impact - A survey of Http unsecure session handling directly leads to unauthorized account access

POORNIMA

experiment - 9

Aim: To use Comodo scan for web vulnerabilities

Basic theory :- Comodo (more often branded under Section 42 c.com) provide an automated web vulnerability Scanning (WVS) service. Unlike local tools like OWASP-ZAP comodo is often used as a cloud-based software as a service to perform black-box testing. It identifies security holes from an outsider's perspective mimicking how an attacker would view your web application over the internet.

Core capabilities :
- malware detection
- SSL/TLS Analysis
- OWASP Top 10 coverage
- Blacklist Monitoring

Required tools
- comodo web inspector (or a similar cloud vulnerability scanner like Acunetix o.s. Qualys).
- target website : a hosted domain or a server with public with public IP

Subject

Page No.

Step-by-step procedure

Step 1:- Account setup and target Addition

- log in to the console / world web Security dashboard.
- enter the URL of the target web applicate

- Verify ownership : you must often prove you own the Web by uploading a specific HTML file to the root directory or adding a TXT record to your DNS config

Step 2 :- Configuring Scan profiles

- Select the scan type : full Scan VS Guided Scan
- Schedule the scan or select 'Scan Now'.
- Result :- the Scanner begins sending a Series of HTTP requests to every page and input field it discovers.

Step 3 :- Monitoring the Scan

- watching the real-time progress on the dashboard.
- the Scanner will categorize findings into categories like critical, High, Medium and low

Step 4 :- Analyzing the web Inspector Report

- once completed, download the pdf or HTML report
- Analyze the executive Summary for a quick health Score.

POORNIMA

-> Review the detailed findings for specific URL paths and technical evidence of the vulnerability.

Step 5 : Remediating and Re-Scanning

- Based on the report's "How to Fix" Section implement Security patches
- Run a follow-up Scan to verify that the status has changed from vulnerable to clean.

Expected Result / Output

1. Vulnerability Summary :- a chart showing the total number of Security issue found.
2. PCI-DSS compliance Status :- a statement indicating if the site meets the requirements for handling credit card data.
3. Inference :- using enterprise-level automated tools allow for constant monitoring and ensure that new vulnerabilities are detected as soon as they appear in the wild.

Subject

No.

Page No.

POORNIMA

Experiment 10:

Objective :- use burp suite for intercepting and testing web traffic

Theory :- Burp Suite is an integrated platform for performing security testing of web applications. the core component is the interacting proxy. which sits between your browser and the target application. that allows you to see, intercept and modify the raw HTTP requests and responses. that are usually hidden from the user.

Key components used in this lab :-
- Proxy
- Repeater
- intruder
- target

Required Tools
- Burp Suite professional / community edition
- Mozilla firefox (often preferred for its independent proxy settings)
- Foxy Proxy extension (optional but very proxy switching)

Step-by-Step Procedure

Step1 :- configuring the Proxy
1. Open Burp Suite and navigate to the proxy > options tab to ensure the listener is active on 127.0.0.1:8080

Page No.

2. in your browser setting, configure the manual proxy to match those settings IP : 127.0.0.1 Port 8080

3 install the burp ca certificate in your browser so you can intercept encrypted HTTPS traffic

Step2 :- intercepting a Request

1. In Burp : go to proxy > intercept and ensure intercept is ON

2. in your browser, perform an action like clicking a link or submitting a search query.

3. The browser will "hang" while Burp holds the request. the request appears in the request in Burp to see. headers, cookies and parameters.

Step3 :- modifying data in transit

1 find a request that carries a parameter you want to test

2 Modify the value directly in the Burp intercept window

3. click forward

4 result :- observe how the web application responds to the modified value.

Step4 : using the repeater for faster testing

1. Right - click a captured request and select Send to Repeater.
2. in the Repeater tab, you can change parameter and click Send multiple time without recapturing the request in the browser.
3. This is used to test for vulnerabilities like IDOR by changing ID number in the URL

Expected Result / output

- Request / Response logs : A history of all intercepted and Showing both the original and modified requests
- Vulnerability Proof :- a demonstration that modifying a request resulted in unauthorized behavior
- inference :- the experimental demonstrates that client - side controls can be easily bypassed by an attacker using a proxy.